

Course Outline: Promises

Title: Promises in JavaScript **Learnpack:** + Promises **Source:** Week 6, Day 16 **Prerequisite:** Sessions (Week 6, Day 16) **Target Audience:** Beginner developers with basic JavaScript and React knowledge, learning frontend data flow **Total Lessons:** 20 **Estimated Total Length:** ~10,000 words **Format:** Mixed (40% hands-on = 8 lessons)

Course Description

Modern web applications constantly talk to the outside world — fetching user data, loading content, saving form results. But JavaScript runs one thing at a time, and waiting for slow network responses without freezing the entire page requires a special tool: Promises.

This course teaches students how to think asynchronously and write code that handles delayed results cleanly. Students will progress from understanding why async code is necessary, to consuming Promises with `.then()`, to writing modern `async/await` syntax, and finally to handling the errors that inevitably arise when working with real-world data.

By the end of this learnpack, students will be equipped to use `fetch()` confidently — the core skill they will expand on in the next lesson, where they'll work with full API communication across all HTTP methods.

Course Philosophy

This course emphasizes:

- Intuition before syntax: students understand *why* Promises exist before learning how to write them
- Progressive layering: `.then()` first, `async/await` second — so students see the evolution, not just the modern shortcut
- Practical grounding: every abstract concept is demonstrated through `fetch` or `LocalStorage`, things students have already touched

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - The Async Problem	3	~1,500
02 - Promise Anatomy	3	~1,600
03 - Consuming Promises	4	~2,100
04 - Real-World Patterns	4	~2,100
05 - Error Handling	3	~1,600
06 - Assessment	2	~1,200
07 - Conclusion	1	~450
Total	20	~11,000

Section 00: Welcome

00.0 Welcome to Promises 📖 (~450 words)

Learning Objectives:

- Understand what this course covers and why it matters
- Connect the idea of Promises to situations students already encounter in the browser

Content Outline:

- What happens when a web app has to wait for something
- Why JavaScript's single-threaded nature makes this interesting
- What students will be able to do by the end of this course

Transitions: This lesson sets expectations and motivation. No code yet — just framing. The next section dives into the actual problem Promises were invented to solve.

Key Principles:

- Async is not optional in modern frontend development
- Promises are a pattern, not just a syntax
- Understanding the problem first makes the solution obvious

Examples:

- Clicking "Load more posts" on a feed — something has to wait
 - A login form that checks credentials — the page can't freeze while waiting
 - A map that fetches location pins — the rest of the UI stays responsive
-

Section 01: The Async Problem

01.0 JavaScript Runs One Thing at a Time 📖 (~500 words)

Learning Objectives:

- Understand what "single-threaded" means in practical terms
- Recognize why asynchronous behavior is necessary in a browser environment
- Distinguish between synchronous and asynchronous code execution

Content Outline:

- The JavaScript event loop in plain language (no deep internals)
- What "blocking" means — and why it's bad for user experience
- The browser's job: rendering, reacting to clicks, AND running your code
- How async operations (timers, network requests) are handed off

Transitions: This lesson builds mental model. Students now understand the *constraint* that makes async necessary. The next lesson shows how developers tried to solve this before Promises existed.

Key Principles:

- JavaScript can only run one piece of code at a time
- The browser handles network requests outside the main thread
- "Async" means "don't wait here — come back when it's done"

Examples:

- A `setTimeout` that "pauses" without actually stopping the page
 - A synchronous `for` loop that freezes the browser tab
 - Clicking a button while an image loads — both can happen because network work is offloaded
-

01.1 Before Promises 📖 (~500 words)

Learning Objectives:

- Understand what callbacks are and how they were used to handle async results
- Recognize the readability problem that comes from deeply nested callbacks
- Appreciate what problem Promises were designed to solve

Content Outline:

- What is a callback? A function passed as an argument to be called later
- Using callbacks for async results: the old `XMLHttpRequest` pattern
- Callback hell: what happens when you need to chain async operations
- Why deeply nested callbacks are hard to read, debug, and maintain

Transitions: Students now see the pain point. The next lesson introduces the tool built to fix it, along with its core vocabulary.

Key Principles:

- Callbacks work but don't scale gracefully
- Nesting async operations creates "arrow-shaped" code that's hard to follow
- Error handling with callbacks is inconsistent and easy to forget

Examples:

- A simple callback: `setTimeout(() => console.log("done"), 1000)`
 - Two chained async operations that create one level of nesting
 - Three chained operations that start to look messy — the "pyramid of doom"
-

01.2 Spotting Async Issues 📖 (~500 words)

Learning Objectives:

- Identify code that has async timing problems
- Predict what value a variable holds at a given moment in async code
- Explain why reading async results synchronously produces `undefined`

Content Outline:

- Reading async code and tracing execution order
- Why `console.log` after a `setTimeout` may not show what you expect
- The classic mistake: trying to use a value before it's ready

Transitions: This hands-on lesson solidifies the mental model before moving on. Students practice *reading* async code rather than writing it — a critical skill. Next, they'll learn how Promises give structure to this chaos.

Key Principles:

- Async code does not execute top-to-bottom like synchronous code
- You cannot read the result of an async operation on the next line
- Predicting execution order is a skill that takes deliberate practice

Hands-On Exercise: Students are given 4 short code snippets with `setTimeout` and basic variable assignments. For each snippet, they must:

1. Write down the order they *think* the `console.log` statements will fire
2. Run the code and compare with their prediction
3. Explain in one sentence why the output differs from synchronous expectations

Code Example:

```
let message = "not yet";
setTimeout(() => {
  message = "ready!";
}, 1000);
console.log(message); // What prints here?
```

Success Criteria:

- Student correctly predicts execution order for at least 3 of 4 snippets
 - Student can explain in their own words why reading async results immediately produces unexpected values
-

Section 02: Promise Anatomy

02.0 States and Lifecycle 📖 (~500 words)

Learning Objectives:

- Name the three states of a Promise and what each means
- Understand that a Promise represents a value that will exist in the future
- Describe the lifecycle of a Promise from creation to settlement

Content Outline:

- What is a Promise? An object that wraps a future value
- The three states: `pending`, `fulfilled`, `rejected`
- Promises are settled once — they cannot go back to pending
- The difference between "resolved" and "fulfilled" (common confusion)

Transitions: Now that students know the vocabulary, the next lesson shows how to create a Promise from scratch and control which state it ends up in.

Key Principles:

- A Promise starts as `pending` and settles exactly once
- `fulfilled` means success; `rejected` means something went wrong
- Once settled, the state never changes — Promises are immutable in that sense

Examples:

- A job application: `pending` (waiting), `fulfilled` (got the job), `rejected` (didn't get it)
 - A delivery order: `pending` (in transit), `fulfilled` (arrived), `rejected` (lost)
 - `fetch()` returns a Promise that starts `pending` and settles when the server responds
-

02.1 Your First Promise 📖 (~550 words)

Learning Objectives:

- Create a Promise using the `new Promise()` constructor
- Use `resolve()` and `reject()` to control the outcome
- Consume the result using `.then()` and `.catch()` (introduced briefly here, detailed in Section 03)

Content Outline:

- Syntax of `new Promise((resolve, reject) => { ... })`
- Calling `resolve(value)` to fulfill with a value
- Calling `reject(reason)` to reject with an error
- Attaching `.then()` to read the result

- Wrapping a `setTimeout` inside a Promise

Transitions: Students have now created and consumed their first Promise. The next lesson shows what goes wrong when Promises are misused — so students recognize bad patterns before they write them.

Key Principles:

- The executor function runs immediately and synchronously
- `resolve` and `reject` are just functions you call when done
- You can only resolve or reject once — subsequent calls are ignored

Hands-On Exercise: Students build three small Promises:

1. A Promise that always resolves with the string `"success"` after 500ms
2. A Promise that always rejects with the string `"something went wrong"` after 500ms
3. A Promise that resolves or rejects based on whether a random number is greater than 0.5

Code Example:

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Here is your data!");
  }, 1000);
});

myPromise.then((result) => console.log(result));
```

Success Criteria:

- All three Promises created correctly and run without errors
- Student can explain the difference between calling `resolve` and `reject`

02.2 Common Promise Mistakes 📖 (~550 words)

Learning Objectives:

- Identify the most frequent Promise anti-patterns
- Understand why wrapping a Promise in another Promise is unnecessary
- Recognize when a function returns a Promise vs. when it swallows one

Content Outline:

- Forgetting to return a Promise inside a `.then()` chain
- Creating unnecessary Promise wrappers around code that already returns a Promise
- Confusing `resolve` with `return` — they are not the same thing
- Calling both `resolve` and `reject` in the same executor

Transitions: Students now know what *not* to do. Section 03 builds on the brief `.then()` introduction from 02.1 and goes deep into consuming Promises in a clean, readable way.

Key Principles:

- If a function already returns a Promise, do not wrap it in `new Promise()`
- Always return Promises inside `.then()` when chaining
- A Promise executor that calls both `resolve` and `reject` — only the first one counts

Examples:

- The "Promise constructor anti-pattern": wrapping `fetch()` in `new Promise()`

- A `.then()` chain that breaks because the inner function doesn't return
 - A real-world scenario where forgetting `return` causes silent failures downstream
-

Section 03: Consuming Promises

03.0 Using `.then()` and `.catch()` 📖 (~500 words)

Learning Objectives:

- Use `.then()` to handle a fulfilled Promise value
- Use `.catch()` to handle a rejected Promise
- Understand that both `.then()` and `.catch()` themselves return Promises

Content Outline:

- `.then(onFulfilled)` — what the callback receives and when it runs
- `.catch(onRejected)` — what it receives and when it runs
- The return value of `.then()` is a new Promise — that's what enables chaining
- `.finally()` — a brief mention for cleanup regardless of outcome

Transitions: Students can now attach handlers to any Promise. The next lesson shows what happens when you chain multiple `.then()` calls together — and why it's cleaner than callbacks.

Key Principles:

- `.then()` only runs when the Promise is fulfilled
- `.catch()` only runs when the Promise is rejected
- Chaining works because `.then()` always returns a new Promise

Examples:

- `.then(data => console.log(data))` — the simplest handler
 - `.then(...).catch(err => console.error(err))` — the standard pattern
 - `.finally(() => setLoading(false))` — cleaning up a spinner regardless of outcome
-

03.1 Chaining Promises 📖 (~550 words)

Learning Objectives:

- Chain multiple `.then()` calls to process data step by step
- Pass transformed values from one `.then()` to the next
- Place a single `.catch()` at the end to handle errors from any step in the chain

Content Outline:

- How return values flow from one `.then()` to the next
- Transforming data step by step without nesting
- Why chaining is cleaner than nested callbacks
- Where to place `.catch()` to catch errors from anywhere in the chain

Transitions: Students can now write clean chains. The next lesson introduces `async/await` — the modern syntax that makes the same patterns even more readable.

Key Principles:

- Each `.then()` receives the return value of the previous one
- A single `.catch()` at the end catches rejections from any step
- Never nest `.then()` inside `.then()` — that's callback hell with extra steps

Hands-On Exercise: Students are given a mock Promise that returns a raw number string (`"42"`). They must build a `.then()` chain that:

1. Parses it to a number
2. Doubles it
3. Formats it as a currency string (`"$84.00"`)
4. Logs the final result

Then add a `.catch()` and test by making the first step reject instead of resolve.

Success Criteria:

- Chain produces correct output at each step
 - `.catch()` correctly intercepts the rejection without crashing
-

03.2 async / await 📖 (~500 words)

Learning Objectives:

- Understand that `async/await` is syntax built on top of Promises — not a replacement
- Mark a function as `async` and use `await` inside it
- Recognize that `await` pauses execution inside the function without blocking the browser

Content Outline:

- What `async` does to a function (it always returns a Promise)
- What `await` does: pauses *within the function* until the Promise settles
- `await` can only be used inside an `async` function
- How `async/await` reads like synchronous code while still being async

Transitions: Students now understand the syntax. The next lesson lets them practice converting `.then()` chains into `async/await` — the most common refactoring they'll do in real projects.

Key Principles:

- `async/await` does not make code synchronous — it makes it *read* like it is
- Every `async` function returns a Promise, even if you don't write `return`
- `await` pauses the function, not the entire JavaScript thread

Examples:

- The same operation written first with `.then()`, then with `async/await`
 - An `async` function that returns a value — and why the caller receives a Promise
 - Using `await` on a `setTimeout` wrapped in a Promise
-

03.3 Rewriting Chains with async/await 📖 (~550 words)

Learning Objectives:

- Convert a `.then()` chain into equivalent `async/await` code
- Use `try/catch` with `async/await` for error handling (introduced here, deepened in Section 05)
- Choose between `.then()` and `async/await` based on readability

Content Outline:

- Side-by-side rewrite: `.then()` chain → `async/await`
- Where `try/catch` replaces `.catch()` in the `async/await` world
- When `.then()` is still preferable (quick one-liners, non-async contexts)

Transitions: Students now have both tools. Section 04 puts them to use in real-world scenarios — starting with the most common async operation in frontend development: `fetch()`.

Key Principles:

- `async/await` and `.then()` are interchangeable — pick the one that reads better
- `try/catch` with `await` is the standard error handling pattern
- Mixing both styles in the same file is fine, but mixing them in the same chain is confusing

Hands-On Exercise: Students take the chain they built in lesson 03.1 and rewrite it using `async/await`. Then they must:

1. Add a `try/catch` block
2. Simulate an error and verify it's caught
3. Write a one-sentence comment explaining why they would choose one style over the other in this case

Success Criteria:

- Rewritten code produces identical output to the chain version
 - `try/catch` correctly catches simulated errors
 - Student's comment reflects genuine reasoning, not just "async/await is newer"
-

Section 04: Real-World Patterns

04.0 Fetching Data with GET 📖 (~500 words)

Learning Objectives:

- Understand that `fetch()` returns a Promise
- Describe the two-step process of reading a fetch response
- Distinguish between a network error and a non-200 HTTP response

Content Outline:

- `fetch(url)` returns a Promise that resolves with a `Response` object
- The `Response` object is not the data — you need to call `.json()` (which is also a Promise)
- Why `fetch()` only rejects on network failure, not on 404 or 500 responses
- Reading `response.ok` to detect non-200 status codes manually

Note: HTTP methods, headers, and authentication are covered in the next learnpack (Week 6, Day 17 — Interacting with APIs). This lesson covers GET-only `fetch()` to demonstrate Promises in a practical context.

Transitions: Students now understand what `fetch()` actually does under the hood. The next lesson puts that into practice with a real API call.

Key Principles:

- `fetch()` is a Promise-based API built into the browser
- Getting the data requires two awaited steps: `fetch()` then `.json()`
- HTTP errors (404, 500) do not automatically reject the Promise — you must check manually

Examples:

- `fetch("https://api.example.com/data")` — what it returns and what `.json()` does
 - Checking `response.ok` before calling `.json()`
 - The difference between a network timeout (rejected) and a 404 (resolved but not ok)
-

04.1 Your First fetch() 📖 (~600 words)

Learning Objectives:

- Write a complete `fetch()` call using `async/await`
- Extract and use data from a real public API response
- Handle the two-Promise chain (`fetch` + `.json()`) correctly

Content Outline:

- Choosing a simple, public, no-auth API for the exercise
- Writing the `async` function, `await fetch()`, checking `response.ok`, `await response.json()`
- Displaying the result in the console, then in the DOM
- Wrapping the whole thing in `try/catch`

Transitions: Students have made their first real network request. The next lesson introduces `Promise.all()` — for when you need multiple fetches to complete before doing something with the results.

Key Principles:

- Always check `response.ok` before calling `.json()`
- Always wrap `fetch()` in `try/catch` — network failures happen
- `await` each step: first the fetch, then the JSON parse

Hands-On Exercise: Using the public API `https://pokeapi.co/api/v2/pokemon/pikachu`, students must:

1. Write an `async` function called `getPokemon`
2. Fetch the data and check `response.ok`
3. Parse the JSON and log the Pokémon's name and base experience to the console
4. Add a `try/catch` and test it by using an invalid URL

Code Example:

```
async function getPokemon() {
  try {
    const response = await fetch("https://pokeapi.co/api/v2/pokemon/pikachu");
    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }
    const data = await response.json();
    console.log(data.name, data.base_experience);
  } catch (error) {
    console.error("Something went wrong:", error.message);
  }
}
```

Success Criteria:

- Function fetches and logs the correct data
- Invalid URL triggers the catch block with a readable message
- Student can explain why there are two `await` statements

04.2 When to Use `Promise.all()` 📖 (~500 words)

Learning Objectives:

- Explain what `Promise.all()` does and when it's appropriate
- Understand that `Promise.all()` rejects if any single Promise rejects
- Distinguish between sequential and parallel async operations

Content Outline:

- The problem: waiting for three unrelated fetches sequentially is slow
- `Promise.all([p1, p2, p3])` — runs them in parallel, resolves when all finish
- What happens if one rejects: the whole `Promise.all()` rejects immediately
- When NOT to use `Promise.all()` : when requests depend on each other's results

Transitions: Students now know when parallelism helps. The next lesson dives into a more nuanced pattern: using the result of a Promise to conditionally resolve or reject — useful for validation logic.

Key Principles:

- Use `Promise.all()` for independent operations that can run at the same time
- `Promise.all()` is all-or-nothing: one rejection cancels the result
- Sequential `await` is correct when each request depends on the previous result

Examples:

- Fetching three different Pokémon simultaneously — good use of `Promise.all()`
- Fetching a user, then using their ID to fetch their orders — sequential, not parallel
- A dashboard that needs data from three endpoints before rendering

04.3 Validating Inside Promises 📖 (~500 words)

Learning Objectives:

- Conditionally resolve or reject a Promise based on runtime logic
- Use validation inside an `async` function to control flow
- Apply this pattern to a realistic input-checking scenario

Content Outline:

- Revisiting the Promise constructor: calling `reject()` based on a condition
- Doing the same with `async/await` : throwing an error inside `async` rejects the returned Promise
- Using this to validate user input before making a network request

Transitions: Students can now validate inside `async` flows. Section 05 completes the picture by focusing entirely on error handling — what to do when things go wrong and how to communicate failures clearly.

Key Principles:

- Throwing an error inside an `async` function rejects its returned Promise
- Validation before fetching saves unnecessary network requests
- Rejecting with a descriptive message makes debugging faster

Hands-On Exercise: Students write an `async` function called `searchPokemon(name)` that:

1. Validates that `name` is a non-empty string — if not, rejects with `"Name is required"`
2. Validates that `name` is at least 3 characters — if not, rejects with `"Name too short"`
3. If valid, fetches from the PokéAPI and resolves with the Pokémon's name and type
4. Test it with an empty string, a 2-character string, and a valid name

Success Criteria:

- Each invalid input triggers the correct rejection message
- Valid input returns the expected data
- Student understands that throwing inside `async` is equivalent to rejecting

Section 05: Error Handling

05.0 Why Async Errors Are Different 📖 (~500 words)

Learning Objectives:

- Explain why a standard `try/catch` cannot catch errors from unhandled Promises
- Describe what an unhandled Promise rejection is and why it matters
- Understand that error handling must be explicitly attached to every Promise chain

Content Outline:

- Synchronous `try/catch` does not reach into Promises
- What happens in the browser when a Promise rejects without a `.catch()`
- The `unhandledrejection` event in the browser
- Why async errors feel "silent" — and why that's dangerous

Transitions: Now that students understand the unique challenge, the next lesson shows the two main tools for catching async errors — and when to use each.

Key Principles:

- Every Promise chain needs either `.catch()` or `try/catch` — not "one of them somewhere"
- Unhandled rejections don't always crash visibly — they can fail silently
- Defensive programming means assuming the network will fail

Examples:

- A `try/catch` around an async call that doesn't use `await` — the error escapes
 - A `.then()` chain with no `.catch()` — what the browser console shows
 - A production bug caused by a silent rejection that went unnoticed
-

05.1 `.catch()` and `try/catch` 📖 (~600 words)

Learning Objectives:

- Apply `.catch()` correctly to a Promise chain
- Apply `try/catch` correctly inside an `async` function
- Choose the right approach based on the code structure

Content Outline:

- `.catch()` placement: at the end of a chain to catch anything above it
- `try/catch` inside `async`: wraps `await` statements
- Using `finally` / `try/catch/finally` to always run cleanup code
- Providing user-facing error messages instead of raw error objects

Transitions: Students can now catch errors. The final lesson in this section shows what "catching" an error wrong looks like — the most common async anti-patterns around error handling.

Key Principles:

- `.catch()` at the end of a chain catches all rejections above it
- `try/catch` with `async/await` is the same concept, different syntax
- Always provide a human-readable message, not a raw stack trace

Hands-On Exercise: Students take the `searchPokemon` function from 04.3 and:

1. Add a `.catch()` version (call the function without `await`, chain `.catch()`)
2. Add a `try/catch` version (call the function with `await` inside an `async` wrapper)
3. For each version, log a friendly message like `"Could not find that Pokémon. Please try again."`

4. Use `finally` to log "Search complete" regardless of outcome

Success Criteria:

- Both versions correctly catch all rejection cases
 - User-facing messages are readable (no stack traces exposed)
 - `finally` block runs in all cases
-

05.2 Silent Failures 📖 (~500 words)

Learning Objectives:

- Identify code patterns that swallow errors without handling them
- Understand why an empty `.catch(() => {})` is an anti-pattern
- Write error handling that is visible, informative, and actionable

Content Outline:

- The empty catch: `catch(err) {}` — the error disappears, the app looks broken
- Catching an error and not re-throwing when it should propagate
- Logging a technical error message to the user instead of a helpful one
- The "callejón sin salida" anti-pattern: showing an error with no next step

Transitions: Students now recognize bad error handling. The assessment section challenges them to apply everything they've learned in a complete, realistic scenario.

Key Principles:

- An empty `.catch()` is worse than no `.catch()` — it hides problems
- Always log errors internally; always show friendly messages externally
- Every error state should offer the user something to do next

Examples:

- `catch(err) {}` — the worst pattern: silent failure, no trace
 - `catch(err) { alert(err) }` — bad: exposes technical details to the user
 - `catch(err) { setError("Couldn't load your data. Try refreshing."); console.error(err); }` — correct pattern
-

Section 06: Assessment

06.0 Building a Promise-Based Fetcher 📖 (~600 words)

Learning Objectives:

- Apply `fetch()`, `async/await`, validation, and error handling together in one function
- Manage loading and error states alongside the data state
- Write code that gracefully handles success, validation failure, and network failure

Content Outline:

- Building a complete function that validates input, fetches data, and handles errors
- Managing three UI states: loading, data, error
- Using `finally` to clear the loading state

Transitions: This lesson serves as practical preparation for the assessment. Students consolidate all concepts before the knowledge check.

Key Principles:

- Real async code always needs to handle three states: loading, success, error
- Validation before fetching is a professional habit, not optional
- The fetcher function should be reusable — specific only in what it fetches, not how it displays

Hands-On Exercise: Students build a `fetchCharacter(name)` function that:

1. Validates `name` is a non-empty string (reject if not)
2. Fetches from `https://pokeapi.co/api/v2/pokemon/{name}`
3. Checks `response.ok` and throws a descriptive error if not
4. Returns an object with `{ name, type, sprite }` from the response
5. Wraps everything in `try/catch/finally`
6. Logs `"Loading..."` at the start and `"Done"` in `finally`
7. Test with: a valid name, an invalid name (empty), and a non-existent Pokémon

Success Criteria:

- All three test cases behave correctly
- `finally` always runs regardless of outcome
- Returned object has the correct shape
- Error messages are human-readable

06.1 Promises Check 🧠 (~600 words)

Learning Objectives:

- Demonstrate understanding of Promise states, syntax, and error handling
- Identify correct and incorrect async patterns
- Apply conceptual knowledge to scenario-based questions

Question Format: Scenario-based (code snippets + interpretation questions)

Questions:

1. A developer writes the following code. What will be logged to the console, and in what order?

```
console.log("A");
const p = new Promise((resolve) => {
  console.log("B");
  setTimeout(() => resolve("C"), 500);
});
p.then(val => console.log(val));
console.log("D");
```

(Expected: A, B, D, C — and student must explain why)

2. A Promise is returned from a function. The function is called but nothing happens — no data, no error. What is the most likely cause? (Expected answer: the returned Promise has no `.then()` or `.catch()` attached, and likely no `await`)
3. A developer wants to fetch data from two unrelated endpoints and combine the results. They write:

```
const a = await fetch(url1);
const b = await fetch(url2);
```

Is there a better approach? What is it and why? (Expected: `Promise.all([fetch(url1), fetch(url2)])` — explains parallelism)

4. Look at this error handler:

```

async function loadUser() {
  try {
    const res = await fetch("/api/user");
    const data = await res.json();
    return data;
  } catch (err) {}
}

```

What is wrong with this code? (Expected: empty catch swallows the error silently — no logging, no user feedback)

5. A `fetch()` call returns a response with status 404. Will the Promise reject automatically? What should the developer do? (Expected: No — `fetch` only rejects on network failure. Developer must check `response.ok` and throw manually)
6. What does `async` do to the return value of a function? Write a one-line example that demonstrates this. (Expected: wraps it in a Promise — `async function greet() { return "hi"; }` returns `Promise<"hi">`)

Evaluation Criteria:

- Questions 1, 5, 6: Test conceptual understanding of how Promises work
- Questions 2, 4: Test ability to recognize anti-patterns
- Question 3: Tests knowledge of `Promise.all()` and when to use it

Score Interpretation:

- 6/6: Ready for the next learnpack (full API communication)
- 4–5/6: Solid foundation; review any missed question before moving on
- Below 4/6: Revisit Sections 03 and 05 before continuing

Section 07: Conclusion

07.0 Your Async Toolkit (~450 words)

Content Outline:

- Course recap: what students accomplished
 - Understood why async code exists and what it solves
 - Learned to create and consume Promises
 - Wrote both `.then()` chains and `async/await` functions
 - Used `fetch()` to pull real data from a public API
 - Applied `Promise.all()` for parallel requests
 - Handled errors with `.catch()`, `try/catch`, and `finally`
 - Recognized the most common async anti-patterns
- Connection to bigger picture
 - Promises are the foundation of all network communication in the browser
 - The next learnpack (Interacting with APIs) builds directly on `fetch()` — now students add POST, PUT, DELETE, headers, and authentication
 - Everything in React data-fetching (custom hooks, `useEffect` with `fetch`) relies on the same patterns learned here
- Next steps and continued learning
 - Practice reading unfamiliar async code — trace execution order before running it
 - Explore `AbortController` for canceling fetches
 - Look into `Promise.allSettled()` for cases where you want all results regardless of failure
 - When you reach React, revisit `useEffect` with fresh eyes — it's just async patterns in a component lifecycle

- Resources for further exploration
 - MDN: Using Promises
 - MDN: async function
 - MDN: fetch API
 - JavaScript.info: Promises, async/await
- Encouragement Async code trips up almost every developer at some point — predicting execution order, managing multiple states, catching errors in the right place. The fact that you've worked through it systematically means you're building real instincts. Trust the mental model: every async operation is a Promise, every Promise needs a handler, every handler needs an error path.

Note: This is conclusion only — no theory, exercises, or assessment.